



AI 발표자료 :

AI 핵심 기능

1. 음식 이미지 인식 모델

1-1 사용한 데이터셋

출처 : AI Hub - 음식 이미지 및 영양정보 텍스트 ([데이터 찾기](#) - AI 데이터찾기 - AI-Hub)



#음식종류 #음식 양 #칼로리 #한식 #이미지

음식 이미지 및 영양정보 텍스트

분야 영상이미지 유형 이미지

구축년도 : 2020 갱신년월 : 2021-12 조회수 : 102,189 다운로드 : 10,171 용량 : 1.60 TB

전체 데이터셋 구조

학습 데이터

- 라벨링데이터(바운딩 박스 위치)
- 원천데이터(음식 이미지)
 - 음식양추정용 이미지는 음식 양별로 분류됨

-

122.음식 분류를 위한 음식종류 및 양에 따른 칼로리 데이터셋(재료, 양념, 완제품 등)

-

01.데이터

-

1.Training

-

라벨링데이터

-

음식양추정

-

양추정_라벨링_TRAIN.zip | 53.20 MB | key: 44896

파일키 복사

다운로드

-

음식분류

-

음식분류_라벨링_TRAIN_1223_add.zip | 445.73 MB | key: 44897

파일키 복사

다운로드

-

원천데이터

-

양추정_이미지_TRAIN_0422_add.zip | 43.44 GB | key: 44898

파일키 복사

다운로드

-

음식분류_이미지_TRAIN_001_1223_add.zip | 27.44 GB | key: 44899

파일키 복사

다운로드

-

합본_음식분류_이미지_TRAIN_002.zip | 31.84 GB | key: 44900

파일키 복사

다운로드

-

음식분류_이미지_TRAIN_003.zip | 28.28 GB | key: 44901

파일키 복사

다운로드

-

합본_음식분류_이미지_TRAIN_004.zip | 30.46 GB | key: 44902

파일키 복사

다운로드

검증데이터

-

2.Validation

-

라벨링데이터

-

양추정_라벨링_VAL.zip | 7.20 MB | key: 44877

파일키 복사

다운로드

-

음식분류_라벨링_VAL_1223_add.zip | 55.81 MB | key: 44878

파일키 복사

다운로드

-

원천데이터

-

양추정_이미지_VAL_0422_add.zip | 5.49 GB | key: 44879

파일키 복사

다운로드

-

01_0422_add.zip | 27.67 GB | key: 44880

파일키 복사

다운로드

-

02_03_05.zip | 29.96 GB | key: 44881

파일키 복사

다운로드

-

04.zip | 30.59 GB | key: 44882

파일키 복사

다운로드

-

06_07_08.zip | 30.19 GB | key: 44883

파일키 복사

다운로드

참고

07. [음식분류 과제] 음식 분류를 위한 음식종류 및 양에 따른 칼로리 데이터셋(재료, 양념, 완제품 등)_ (1).pdf

이 중 40개의 클래스에 해당하는 데이터셋을 학습시킴. 400종의 데이터를 다 학습시키기엔 일정상 무리였다.

2026.2.10일 추가

400종 학습 완료

1-2 음식 이미지 분류 모델 : YOLO

1) 사용한 모델 : YOLO 26

출처 : ([Ultralytics YOLO26](#) - Ultralytics YOLO 문서)

2) YOLO26를 사용한 가장 큰 이유

: 이전 버전 대비 CPU환경에서도 구동이 잘 되기 때문

- 최대 43% 빨라진 CPU 추론

특히 엣지 컴퓨팅에 최적화된 YOLO26은 훨씬 빠른 CPU 추론을 제공하여 GPU가 없는 디바이스에서도 실시간 성능을 보장합니다.

>> 즉 모바일 환경에서 서비스 하는 우리의 환경에 최적화 되어 있기 때문에 선택함. 물론 fastapi 자체는 aws 환경에서 배포를 하긴 하지만 gpu 서버에서 굳이 연산을 하지 않아도 되기 때문에 서버 운용비용의 절감 효과가 있을 것으로 예상

YOLOv3에서는 NMS (Non-Maximum suppression, 박스 제거)과정이 필수적이었습니다. 왜냐하면 YOLO v3는 수많은 박스를 만들고 그 과정에서 가장 높은 확률을 가지는 박스를 제외한 나머지 박스를 지워야 했기 때문입니다. 이 과정이 추론 속도에 영향을 미칩니다. 실제로 YOLOv3의 추론 속도는 Titan X 환경의 320 x 320 해상도에서 22ms의 추론속도를 가졌습니다. LearnOpenCV에 의하면 YOLOv3-Tiny의 CPU 추론 시간은 150ms를 기록하였습니다.

반면 YOLO26은 NMS-Free 모델로 박스를 제거하지 않고 바로 추론을 실시하기 때문에 Latency에 있어서 유리함을 가집니다. YOLO26은 CPU 환경에서 87.2ms의 응답속도 (YOLO26 nano, ONNX 런타임 기준)를 기록하였습니다.

또한 YOLO26에서는 표준 해상도가 640 x 640 사이즈로 향상되었기 때문에 YOLOv3보다 높은 해상도를 가지는 이미지에서 더욱 빠른 추론을 수행할 수 있습니다.

저희의 실행환경에서는 평균적으로 106ms를 기록하여 v3 응답 대비 29.3% 개선할 수 있었습니다.

MAP 50-95 수치 비교

YOLOv3: 33.0%

YOLO26: 28.6%

3) 커스텀 데이터셋으로 YOLO 학습 시킨 방법

1. AI Hub에서 다운 받은 학습용 데이터를 다운 받은 후 모든 파일을 구글 드라이브에 업로드함

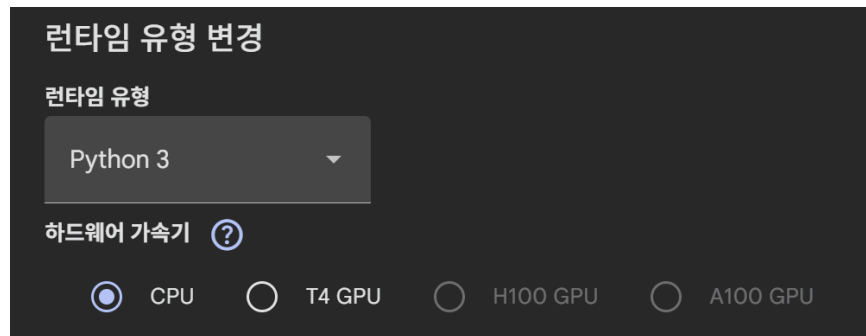
공유 문서함 > Food_Data > yolo > yolo_train_data ▾

유형 ▾ 사람 ▾ 수정 날짜 ▾ 출처 ▾

이름 ↓

	TRAIN_049.zip	
	TRAIN_048.zip	
	TRAIN_047.zip	
	TRAIN_046.zip	
	TRAIN_045.zip	
	TRAIN_044.zip	

2. 구글 코랩 환경(구글 Tpu-T4)로 학습 시키기 위해 Colabatory에서 학습용 모델 생성



>> 이후 서술하겠지만 T4환경에서 안정적인 배치사이즈(한번에 처리하는 이미지 개수)를 높여 정확도를 높이기 위함

+400종의 클래스를 학습시킬때 A100GPU 환경에서 진행함

학습용 모델 파일

[SH_YOLO \(2\).ipynb](#)

학습용 주피터 노트북 구조

- > 데이터 전처리
 - ↳ 숨겨진 셀 1개
- > 라벨 압축 해제
 - ↳ 숨겨진 셀 1개
- > 클래스 추출, 데이터 셋 처리
 - ↳ 숨겨진 셀 1개
- > 학습
 - ↳ 숨겨진 셀 1개
- > 플롯 확인하기
 - ↳ 숨겨진 셀 1개
- > 정답률 확인
 - ▶ ↳ 숨겨진 셀 1개

데이터(이미지) 전처리 방법

하나의 압축 폴더에는 8개의 음식 클래스를 위한 데이터셋이 있고 각 1000장씩 존재하지만 우리의 컴퓨팅 환경이 제한됨에 따라 500개만 학습시키기로 함. 또한 YOLO26의 입력 사이즈가 640이기 때문에 640사이즈로 리사이징함.

```
# 5. 전처리 파라미터
SAMPLE_LIMIT = 500    # 샘플 수
IMG_SIZE = 640        # 이미지 사이즈 (픽셀)
VAL_RATIO = 0.1       # Valid 데이터 셋 비율
# =====
```

학습모델 구조

```
model = YOLO('yolo26s.pt') # Detection 모델
```

>> YOLO26 samll 버전 사용, Fine Tuning

>> 이후 다양한 데이터를 반영하기 위해 YOLO26 medium 사용

```
# 학습 시작
model.train(
    data=yaml_path,      # 위에서 만든 data.yaml 경로
    epochs=100,          # 실제 20epoch에서는 정확도가 30~35%사이로 나타났기 때문에 더 늘릴 필요가 있었다.
    imgsz=IMG_SIZE,      # 224 x 224 size
    batch=32,            # colab환경과 데이터의 전처리 결과물을 확인했을 때 32로 해도 GPU에 무리가 없을 것이라고 판단하였다.
    patience=3,          # 일정 epoch구간동안 개선이 없으면 조기종료 (3번까지는 참는다.)
    dropout=0.1,         # 일정 부분은 뉴런을 꺼서 과적합 방지
    weight_decay=0.0005, # 모델이 너무 복잡해지면 안된다.

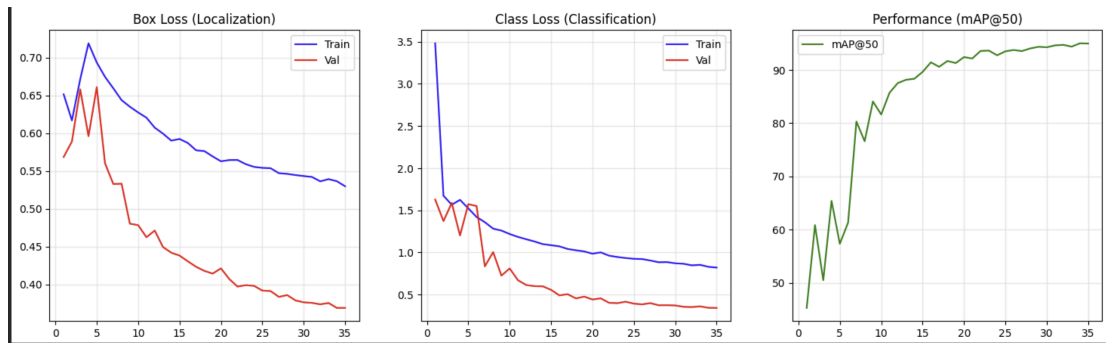
    hsv_h=0.015,         # 색조(색깔) 살짝 변경
    hsv_s=0.7,           # 채도(진하기) 변경
    hsv_v=0.4,           # 명도(밝기) 변경
    mosaic=1.0,          # [팔살기] 사진 4장을 바둑판처럼 합쳐서 학습 (작은 물체 잘 찾게 됨)
    mixup=0.1,           # 사진 2장을 투명하게 겹쳐서 학습 (0.1은 10% 확률)

    project=PROJECT_PATH,
    name=RUN_NAME,
    exist_ok=True
)
```

>> 기본적인 학습 파라미터 설정 + 증강효과까지 넣은 모습

학습 결과

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	640: 100%	558/558	1.2s/it	11:04
1/100	9.35G	0.6516	3.482	0.01453	29	mAP50	mAP50-95): 100%	31/31	1.0it/s	29.8s
	Class	Images	Instances	Box(P	R	0.453	0.382			
	all	1983	3966	0.45	0.465					
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	640: 100%	558/558	1.1s/it	10:05
2/100	9.45G	0.617	1.673	0.01357	39	mAP50	mAP50-95): 100%	31/31	1.3it/s	23.0s
	Class	Images	Instances	Box(P	R	0.609	0.519			
	all	1983	3966	0.604	0.557					
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	640: 100%	558/558	1.0it/s	9:12
3/100	9.54G	0.6715	1.567	0.01473	29	mAP50	mAP50-95): 100%	31/31	1.4it/s	22.0s
	Class	Images	Instances	Box(P	R	0.505	0.417			
	all	1983	3966	0.518	0.494					
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	640: 100%	558/558	1.0it/s	9:08
4/100	9.47G	0.7191	1.625	0.01571	29	mAP50	mAP50-95): 100%	31/31	1.4it/s	22.4s
	Class	Images	Instances	Box(P	R	0.654	0.552			
	all	1983	3966	0.614	0.62					
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	640: 100%	558/558	1.0it/s	9:13
5/100	9.47G	0.6937	1.524	0.01505	32	mAP50	mAP50-95): 100%	31/31	1.4it/s	21.4s
	Class	Images	Instances	Box(P	R	0.574	0.473			
	all	1983	3966	0.565	0.566					



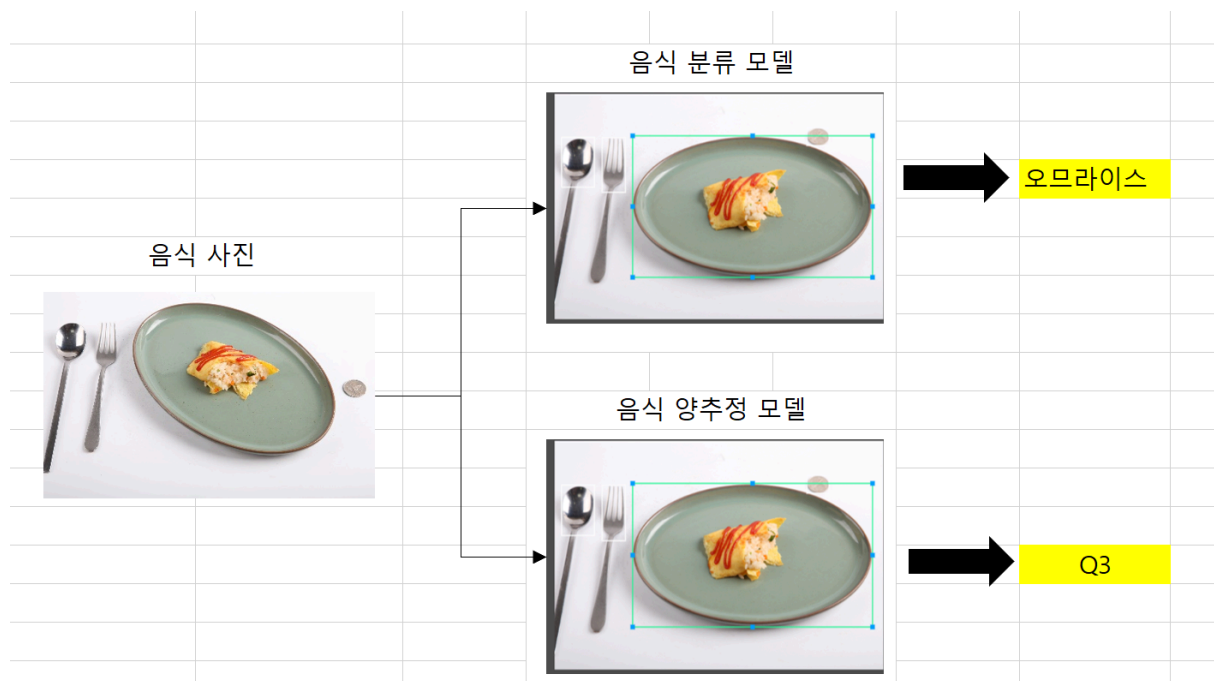
해석 : map50 : 물체와 박스가 50% 이상 겹치면 "정답!"이라고 인정해 주는 기준에서 매긴 평균 점수

1-3 음식 이미지 중량 분류 모델 : RESNET

음식을 찍으면 디비에 저장되는 정량기준 → 기록된 양만 가져올 수 있는데, resnet을 붙이면 무게 가중치를 넣어서 좀 더 정확한 중량을 확인할 수 있어 정확도 상승 (89% → 신뢰도)

직접 우리가 resnet으로 학습을 시키진 않았음

이미 만들어진 가중치 파일을 가지고 단순히 사용하기만 함



내부 구현 구조 : yolo에서 이미지 분류를 하면 resnet에서 추가로 연산하기 위해 이미지 사이즈를 다시 224*224 크기로 조정

```
def _load_resnet_model(self, path):
    try:
        model = models.resnet18(weights=None)
        model.fc = nn.Linear(model.fc.in_features, 5) #
Q1 ~ Q5

        # 가중치 파일 로드 (CPU/GPU 자동 매핑)
        checkpoint = torch.load(path, map_location=self.device, weights_only=False)

        # 딕셔너리 구조에 따라 유연하게 로드
        state_dict = None
        if isinstance(checkpoint, dict):
            if 'model_ft' in checkpoint:
                state_dict = checkpoint['model_ft'].state_dict() if isinstance(checkpoint['model_ft'], nn.Module)
            else checkpoint['model_ft']
            elif 'state_dict' in checkpoint:
                state_dict = checkpoint['state_dict']
            else:
                state_dict = checkpoint
        elif isinstance(checkpoint, nn.Module):
            state_dict = checkpoint.state_dict()

        if state_dict:
            model.load_state_dict(state_dict, strict=False)
        else:
            print("⚠ ResNet 가중치를 제대로 불러오지 못했습니다. (빈 모델 사용)")

        model.to(self.device).eval()
        return model
    except Exception as e:
```

```
print(f"❌ ResNet 로드 실패: {e}")
return None
```

1-4 백엔드와의 연결

1) API호출

백엔드에서 음식이미지를 사용자에게 받아와서 db에 저장하고 Ai 이미지 분석 요청을 하면 Ai에서는 연산을 마치고 아래 형태와 같은 연산 결과를 보내줌

```
{
  "result": [
    {
      "food_name": "01015006",
      "confidence": 0.8,
      "quantity": "Q4",
      "box": {
        "x_min": 0.0008664607885293663,
        "y_min": 0.31222933530807495,
        "x_max": 0.8304023742675781,
        "y_max": 1.0
      }
    }
  ],
}
```

2. 영양성분 추출

2-1 백엔드 Spring에서 탄수화물 중량 계산

1) 위의 과정에서 받은 결과값으로 분류된 음식코드와 질량을 바탕으로 사용자가 섭취한 탄수화물량을 계산

음식분류 AI 데이터 영양DB.xlsx

음식분류 AI 데이터 영양DB_활용가이드.xlsx

음식 양 분석 계산식 = '음식 중량' X '음식 양 분석 결과'

즉, 알고리즘 결과 '오므라이스' 음식에 해당하는 음식 중량과 Q3에 해당하는 75%를 곱하여 음식 양 분석

오므라이스 음식 양 = 450 g X 0.75 = 337.5 g

음식명	중량(g)	에너지(kcal)	탄수화물(g)	당류(g)	지방(g)	단백질(g)	칼슘(mg)	인(mg)	나트륨(mg)	칼륨(mg)	마그네슘(mg)	철(mg)	아연(mg)	콜레스테롤(mg)	트랜스지방(mg)
일반비빔밥	500.00	703.00	95.68	3.44	25.28	22.60	96.80	289.33	1234.09	742.51	55.80	4.75	4.16	40.20	0.00
전주비빔밥	450.00	662.19	92.99	3.44	13.03	15.84	65.04	236.12	1281.30	450.11	10.84	3.24	2.86	286.94	0.00
삼선볶음밥	400.00	683.62	113.43	5.09	16.84	19.26	119.37	249.11	1155.81	640.97	29.67	4.52	3.57	119.13	0.00
새우볶음밥	400.00	634.52	91.62	0.00	17.91	24.12	81.82	341.30	1124.63	710.66	17.68	4.15	2.82	127.20	0.00
알밥	400.00	606.54	92.10	0.00	3.45	15.16	68.60	235.40	1647.87	510.41	7.40	3.56	3.30	39.00	0.00
산채비빔밥	400.00	495.89	89.66	0.52	10.99	10.66	63.10	160.29	1060.78	509.95	38.81	3.53	2.05	0.00	0.00
오므라이스	450.00	684.99	101.62	0.00	19.75	23.15	81.56	341.26	1123.24	788.38	33.71	4.13	3.67	605.63	0.00
육회비빔밥	450.00	661.41	91.74	2.58	17.54	32.00	86.38	404.74	1126.60	793.74	33.58	5.97	6.53	241.65	0.00
해물볶음밥	400.00	659.23	85.19	0.00	23.71	22.68	77.50	305.55	951.03	555.36	24.20	4.13	3.08	362.40	0.00
얼루비빔밥	400.00	445.66	90.03	1.72	3.16	12.99	91.06	167.96	718.39	708.68	1.29	3.91	2.65	12.80	0.00

```
class PredictionInput(BaseModel):
    user_id: int
    glucose: float          # 현재 혈당
    carbs_in: float         # 현재 섭취량
    timestamp: str          # 현재 시간

    # 백엔드가 DB 보고 판단/조회한 값들
    has_enough_data: bool = False
    weight: float = 70.0
    height: float = 170.0
    is_t2d: bool = True
```

2) 계산된 탄수화물 섭취 양을 db에 저장

저장된 데이터는 이후 사용자별 혈당 예측 모델에 사용됨

3. 혈당 예측 모델

3-1 사용한 데이터셋 및 전체적인 플로우

1. 데스크 api에서 발행한 가짜 데이터셋의 events테이블과 evgs테이블에 있는 모든 데이터를 db에 저장
2. 회원가입시 키, 몸무게 받아와야 함(수학적 모델에 사용하기 위함)
3. 여기까지 진행하게 되면 유저 음식 히스토리는 DB에 음식이미지와 제출 시간도 함께 같이 저장하게 됨.(초 단위 저장x)
4. 혈당예측모델은 사용자의 cgm 착용 데이터가 4일미만일 경우 수학적 모델을, 4일 이상일 경우 회귀분석 머신러닝모델(xgboost)을 사용해서 연산할 예정
5. 단 xgboost의 경우 동시에 여러 명의 사용자가 연산을 위한 개인별 모델 학습을 진행할 때(사용자별 모델을 24번 학습시켜야 하기 때문) 서버 과부하의 위험이 있기 때문에 사용자가 접속하지 않은 시간에 백엔드에서 하루에 한번 ai서버로 api요청을 보내서 자동으로 업데이트하게 할 예정(api/v1/updates)

3-2 백엔드에서 혈당예측모델을 호출 시 보내는 데이터

1)사용자의 데이터가 4일 미만일 경우 : 논문 기반 수학적 모델링

2)사용자의 데이터가 4일 이상일 경우 : xgboost 머신러닝 모델(회귀분석 기반)

- Xgboost모델 학습용 데이터 : 사용안할때 자동 업데이트시(새로 수집된 데이터를 반영시켜 모델을 갱신하기 위함) 백엔드에서 하루에 한번 자동 호출

```
class GlucoseEntry(BaseModel):
    glucose_id: Optional[int] = None
    user_id: int
    sensor_id: Optional[int] = None
    value: float          # ★ 핵심: 혈당 값
    trend: Optional[str] = None
    measured_at: str      # ★ 핵심: 측정 시간
```

- 실시간으로 4일 이상 사용한 사용자가 실시간 혈당 예측을 요청할 때 (이미 만들어진 모델에 연산만 요청)

```
class EventEntry(BaseModel):
    value: float          # ★ 핵심: 탄수화물 양
```

measured_at: str # ★ 핵심: 섭취 시간

3-3) 수학적 모델

1) 기본 혈당 모델

Bergman의 최소모델에 의하면 혈당은 최소한 다음의 두가지의 인자에 의해서 결정될 수 있습니다.

- 포도당의 흡수 속도
 - 개개인에 따라서 포도당을 흡수하는 속도가 다릅니다.
 - 탄수화물의 GI에 따라 흡수되는 포도당의 속도가 다를 수 있습니다.
 - 혈장 인슐린의 농도가 높을수록 평균적인 세포의 포도당 흡수속도가 빠릅니다.
 - 혈장 포도당의 농도가 높을수록 평균적인 세포의 포도당 흡수속도가 빠릅니다.
 - 기저 혈당이 높다면 평균적으로 나타나는 기울기 상승이 가파릅니다.
- 인슐린의 지연 시간
 - 개개인에 따라서 인슐린의 활성화 시간과 인슐린 저항성이 다르게 나타납니다. (인슐린 저항성과 연관이 깊습니다.)
 - 기저 인슐린 농도에 의하여 체내에서 분비되는 인슐린의 양이 달라질 수 있습니다. (혈장 인슐린의 농도와 관련이 깊습니다.)

저희의 서비스는 혈당을 엄격하게 관리하지 않아도 되는 사람들을 대상으로 음식의 섭취유무를 제공하므로 포도당의 흡수속도와 그로 인해 결정되는 최고 혈당치에 주목할 필요가 있었습니다. 따라서 인슐린의 지연시간은 비교적 단순하게 정상인, 2형 당뇨병 유병자로 구분하여 다른 하강 기울기 가중치를 부여하였고, 포도당의 흡수 속도와 관련한 영향력을 높여 혈당을 모델링합니다.

(Minimal Model, Richard N. Bergman), 기초 혈당 모델링

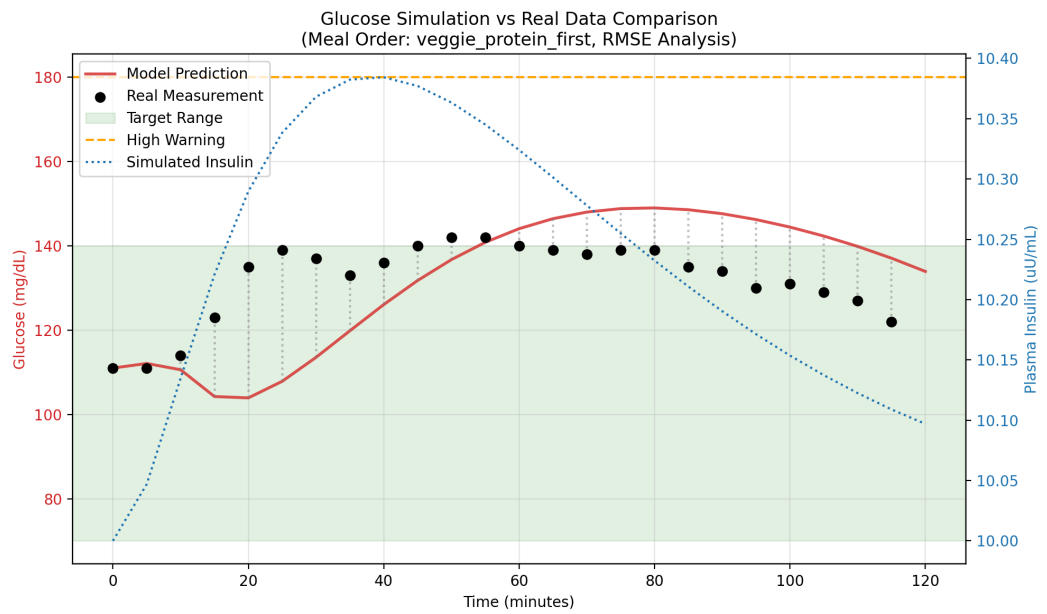
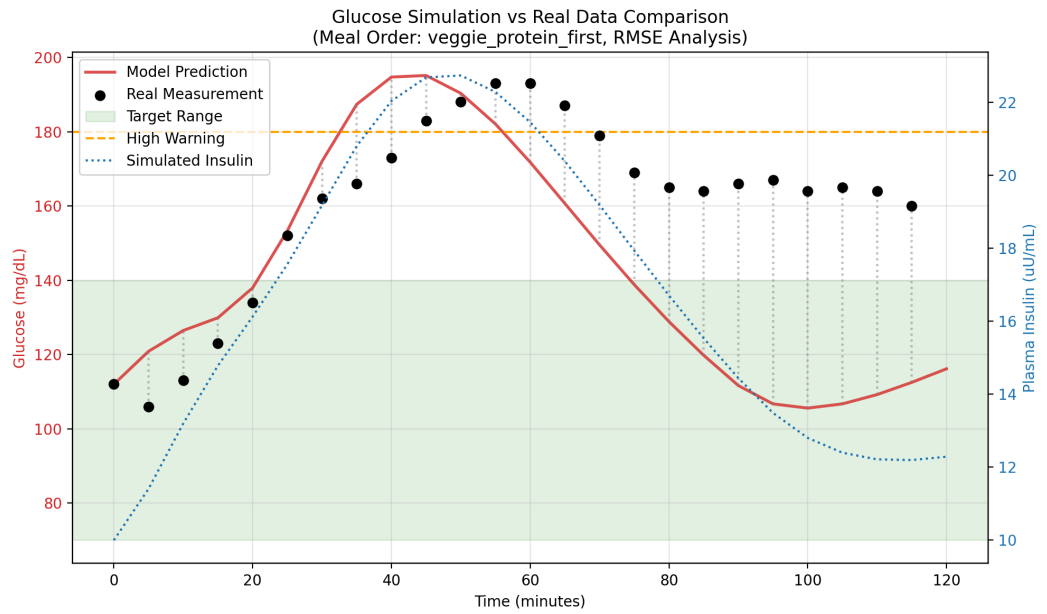
(Realistic Stochastic Model of Gastric Emptying, Jiraphat Yokrattanasak), 음식물 섭취 순서와 위 배출속도의 관계

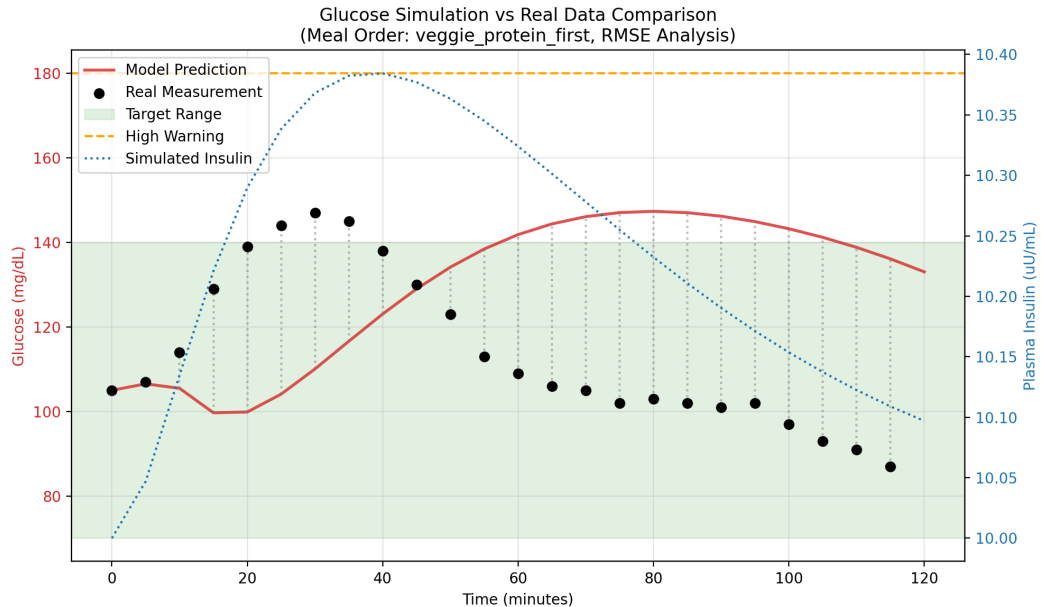
(Effects of Protein Preload on Gastric Emptying, Glycemia, and Gut Hormones After a Carbohydrate Meal in Diet-Contolled Type 2 Diabetes, Jing Ma, Julie E. Stevens), 식사순서에 따른 혈당 상승속도 변화

2) 수학적 모델의 선정

수학적 모델은 두가지 모델을 가정하여 비교를 진행하였습니다.

- 논문에 기반한 수학적 모델 구성
 - 포도당과 인슐린의 농도의 관계와 위 배출속도와 섭취한 식이섬유의 작용과 관계되는 포도당 흡수 속도를 고려하여 모델을 설계하였습니다.
 - 해당 모델의 예측률은 다음과 같았습니다.
 - $N = 22$
 - RMSE의 평균: 27.94mg/dL (최소 RMSE 12.04mg/dL, 최대 RMSE 52.86mg/dL)
 - MAPE의 평균: 25.51% (최소 MAPE 12.95%, 최대 MAPE 55.81%)
 - 혈당 최고치의 오차 평균(시점 관계없이 최댓값 비교): +25.87mg/dL (최소 : -0.3mg/dL, 최대 : +42.9mg/dL)
 - 혈당 최고치의 오차율 평균: +13.57% (최대 +41%, 최소 0.2%)
 - 해당 모델의 문제점은 다음과 같았습니다.
 - 혈당의 최고치를 예측하는 시점이 늦다는 문제점이 존재합니다. : 추가적인 음식물의 섭취를 배제하였기 때문에 혈당의 추가적인 상승을 예측하지 못한다는 문제점이 존재합니다.
 - 혈당의 최고치를 중요하게 생각하게 된다면 RMSE의 평균이 훨씬 높아지는 경우가 발생했습니다. 특히 기저혈당이 높거나, 섭취한 탄수화물의 양이 많아질 수록 혈당의 상승을 과대평가하는 경향을 확인할 수 있었습니다.
 - 인슐린의 분비가 가능하다는 전제하에 가정한 모델입니다. (분비량 가중치를 0.05라고 가정)
 - 정상적인 혈당 추이와 비교했을 때 예측 혈당이 낮아지는 경우가 존재했습니다. (0.105)





3) 기율기의 선정

일반적으로 포도당의 농도는 포도당의 효율성과 인슐린의 생성과 작용 계수, 섭취한 탄수화물의 당지수에 의해서 결정되지만, 해당 모델에서는 가벼운 당뇨병 2형 유병자나, 당뇨병 관리가 필요한 당뇨병 위험군이 타겟이기 때문에 정상인에 가까운 기율기로 계수를 채택할 수 있었습니다. 특히 2형 당뇨병에 큰 영향을 미치는 부분(비만 여부에 의한 인슐린 저항성, 섭취하는 탄수화물의 종류 등)은 개별 파라미터로 분리하여 기율기에 가중치를 부여하여 추정하였습니다.

4) 질문 방어

특히 혈당은 다음과 같은 경우에 높아질 수 있습니다.

1. 탄수화물의 특성 : 체액에 담길 수 있는 탄수화물의 용적이 늘어난다면 혈당의 기율기는 일관되게 상승합니다.
2. 비만 여부와 당뇨병에 따른 인슐린 저항성 : 2형 당뇨병의 80%이상은 비만에 의해서 발생하고, 2형 당뇨병은 베타세포가 파괴되지 않은, 인슐린 저항성이 높아져 혈당을 효과적으로 줄일 수 없는 상태를 의미합니다.
3. 기저 혈당 : 기저 혈당이 높을 수록 반응하는 혈장 인슐린 양인 $X(t)$ 가 낮게 측정되는 경향이 있습니다. (혈장 인슐린의 농도에 비해서 활성화된 인슐린의 농도가 낮은 상태)

=====기존 특이 사항
=====

1. 운동의 여부를 판단하지 않는 이유, 식사 순서와 식이섬유의 영향력 배제
 - a. 운동은 일시적으로 인슐린 저항성을 우회할 수 있는 수단 중 하나입니다. GLUT4 수송체(인슐린의 자극을 받아서 세포막으로 이동하면 포도당을 세포에 흡수시키게

하는 단백질)를 사용하여 혈류내의 포도당을 흡수하는 과정을 거치는 것이 아닌 근육 수축을 통한 AMPK활성화(에너지 부족시 ATP 활성화)를 통해서 인슐린 외의 경로로 포도당을 흡수하기 때문입니다. 하지만, 서비스는 사용자가 어느 정도의, 어떤 운동을 했는지 모르고 사용자 또한 객관적인 운동의 강도를 판단할 수 없으므로, 운동을 하지 않았다는 가정하에 모델을 구성하였습니다.

- b. 식사순서에 의해서 위가 배출하는 음식물의 배출 속도를 정의할 수 있습니다. 식사의 순서를 낮추면 포도당이 혈류에 유입되는 속도를 늦출 수 있어 인슐린의 유입속도와 균형을 맞출 수 있습니다. (이는 식이섬유의 작용과 GLP-1 분비와 유의미한 관계가 있습니다. GLP-1은 식욕 억제와 체내 열량 소비의 증가와 관련이 있는 호르몬입니다.)

2. 나트륨과 혈압이 혈당에 미치는 영향 배제

- a. 일반적으로 당뇨병 환자의 혈압은 높은 것으로 추정되고 있으나, 당뇨가 진행되다 보면 고혈당에 의해서 콩팥의 기능이 떨어지기 시작하며 혈압의 관리가 어려워지게 됩니다. 따라서 당뇨병환자가 고혈압관리를 유지하는 것은 중요합니다. 하지만 본 수학적 모델에서는 음식과 혈당, 개인의 상승 기울기 가중치만으로 평가하기 때문에 직접적인 인과관계가 없다고 판단하였습니다.

3-4) XGBoost 회귀모델 (영록님)

1) XGBoost란?



개념 : XGBoost(eXtreme Gradient Boosting)는 경사 하강법을 활용하는 지도 학습 부스팅 알고리즘인 그레이디언트 부스트 의사결정트리를 사용하는 분산형 오픈 소스 머신 러닝 라이브러리

회귀모델이면서 의사결정트리 기반으로 한 머신러닝 모델

DEXCOM 데이터를 LSTM이랑 XGBOOST 비교를 해서 → 예측 모델에서 잔차제곱평균(MAP)을 비교를 해서

오차 낮게 나와야한다. / 학습 시간이 좀 더 있다.

사용자별로 모델을 만들어야하는데 연산 빠르고 학습 시간이 적게걸린다.

2) 혈당 예측 데이터셋의 특수성

사용자의 혈당 수치 변화는 시계열 데이터라고 볼 수 있다. 시계열 데이터는 시간에 따라 변화하는 데이터를 의미한다. xgboost의 특성상 하나의 포인트를 예측하는건 잘한다. 즉 5분 뒤의 데이터는 예측을 잘하지만 그 5분 뒤의 데이터를 반영해서 다음 5분을 학습해서 처음 지점에서 10분 뒤의 데이터를 학습하는 구조에서는 예측률이 낮았다. 즉 시계열 데이터의 특성상 xgboost로 예측을 진행할 경우 과거 정보가 희석되는 구조였다. 그래서 진행한 방법이 식사한 지점을 시작으로 식후 2시간까지 예측을 진행하기 위해 2시간(120분)을 5분 단위로 나누어서 24개의 포인트로 나누고 각 지점마다 xgboost회귀모델을 만들어서 각 포인트에 대한 회귀예측 하는 기법으로 진행하였다.

식후 2시간까지 예측을 진행하기 위해 2시간(120분)을 5분 단위로 나누어서 24개의 포인트로 나누고 각 지점마다 xgboost회귀모델을 만들어서 각 포인트에 대한 회귀예측 하는 기법으로 진행하였다.

→ 모델을 튜닝을 어떻게 했는지 내용

3) 데이터 전처리

앞서 호출했던 raw data를 예측 모델에 사용할 수 있게 전처리를 진행했다.

전처리 후 결과

```
feature_cols = [  
    'glucose', 'carbs_in', 'COB',  
    'hour_sin', 'hour_cos',  
    'glucose_lag_15m', 'glucose_lag_30m', 'glucose_lag_60m', 'glucose_lag_120m',  
    'glucose_roc_15m', 'past_velocity_15m', 'past_accel_15m'  
]
```

(변수 설명 예정)

4) 핵심 기법 : MultiOutputRegressor

```
model = MultiOutputRegressor(xgb.XGBRegressor(n_estimators=
1000, n_jobs=-1, random_state=42))
```

MultiOutputRegressor는 앞서 설명했던 24개의 모델을 사용하기 위해 사용한 기법이다. 독립적인 여러 개의 예측 모델을 하나로 묶어 관리함으로써, 복잡한 다중 목표 데이터를 쉽고 효율적으로 예측할 수 있게 해주는 강력한 래퍼(Wrapper) 도구다.

→ 하나의 포인트에 대한 하나의 모델이 있다. 어떤 모델을 썰는지 언급

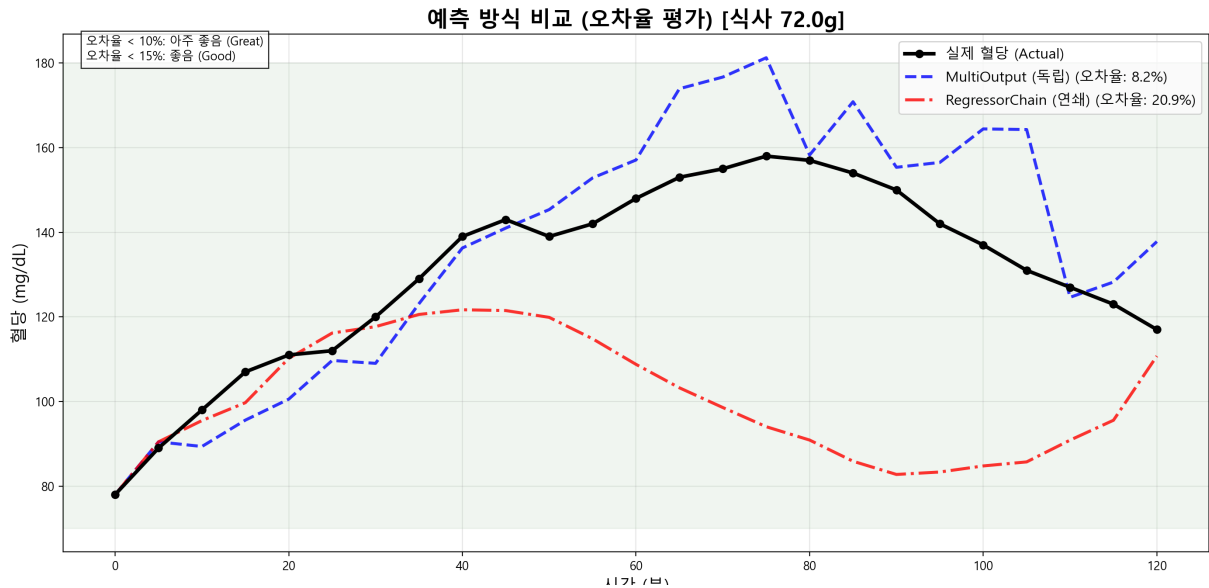
옵션명	값	설명 (발표용 멘트)
n_estimators	1000	"총 1,000개의 의사결정 나무를 만들겠다는 뜻이야. 나무가 많을수록 더 꼼꼼하게 공부하지만, 너무 많으면 시간이 오래 걸릴 수 있어."
n_jobs	-1	"하나의 나무를 만들 때도 컴퓨터의 모든 성능을 다 빌려 쓰겠다는 의미야. 속도를 최대한으로 높이기 위한 설정이지."
random_state	42	"인공지능도 공부할 때 무작위성이 들어가거든. 42 라는 숫자로 고정해두면, 누가 언제 실행해도 똑같은 성적이 나오게 해줘. (재현성 확보)"

개별 학습:

- 1번 모델은 '5분뒤'만 공부하게 시켜.
- 2번 모델은 '10분뒤'만 공부하게 시켜.
- 3번 모델은 '15분뒤'만 공부하게 시켜.

8일 학습치 데이터 비교

예시)



설명 포인트 : 파란색 선(multioutputregressor적용)은 각 포인트의 오차폭이 크더라도 다음 포인트에 영향을 미치지 않기 때문에 실제 혈당 변화 추세에 따라가려 하지만 하나의 모델을 사용한 경우(연쇄) 한번 잘못 예측 시작하면 쪽 벗어나게 된다

5) 모델 자동 업데이트

사용자가 실시간으로 사용하기에 한개의 xgboost모델을 그때마다 학습시켜서 결과값을 도출하는 것은 평균5초 정도 걸리는 반면 현재 우리가 사용한 방법의 경우에는 동시에 24개의 xgboost모델의 연산 값이 필요한 상황이다. 그때마다 모델을 학습하고 연산하게 되면 시스템 규모가 작은 지금은 몰라도 나중에 규모가 커졌을 때 수많은 사용자가 동시에 학습과 연산을 동시에 요청하게 되면 서버 과부하는 필연적이다. 그래서 우리 서비스는 잠깐의 점검시간을 통해 매일 00시에 실제로 우리 서비스를 사용하고 있는 사용자에게 한해 예측 모델을 자동으로 갱신하게 설계했다.

→ 사용자 요청에 의해서 매번 모델 학습시키면 연산하면 대답이 오래걸림 → 이용하는 사용자의 데이터를 반영해서 당일 00시 ~ 최대 3개월 전까지 모델 업데이트

내부 구현

```
def train_model_logic(request_data):
    user_id = request_data.user_id
    model_path = get_user_model_path(user_id) # ★ 사용자별
    경로 확인
```

```

print(f"🔄 [Update] User {user_id} 모델 업데이트 중...")

# 1. 병합
df = merge_data_streams(request_data.glucose, request_data.events)

# 2. 데이터 부족 -> 기존 모델 삭제 (오염 방지)
if len(df) < MIN_TRAIN_DATA:
    if os.path.exists(model_path):
        os.remove(model_path)
    return {"status": "switched_to_math", "message": "데이터 부족"}

# 3. 전처리
X, y, weights = preprocess_features(df, is_training=True)

if len(X) == 0:
    return {"status": "error", "message": "유효 데이터 없음"}

# 4. 학습 (Fit)
model = MultiOutputRegressor(xgb.XGBRegressor(n_estimators=1000, n_jobs=-1, random_state=42))
model.fit(X, y, sample_weight=weights)

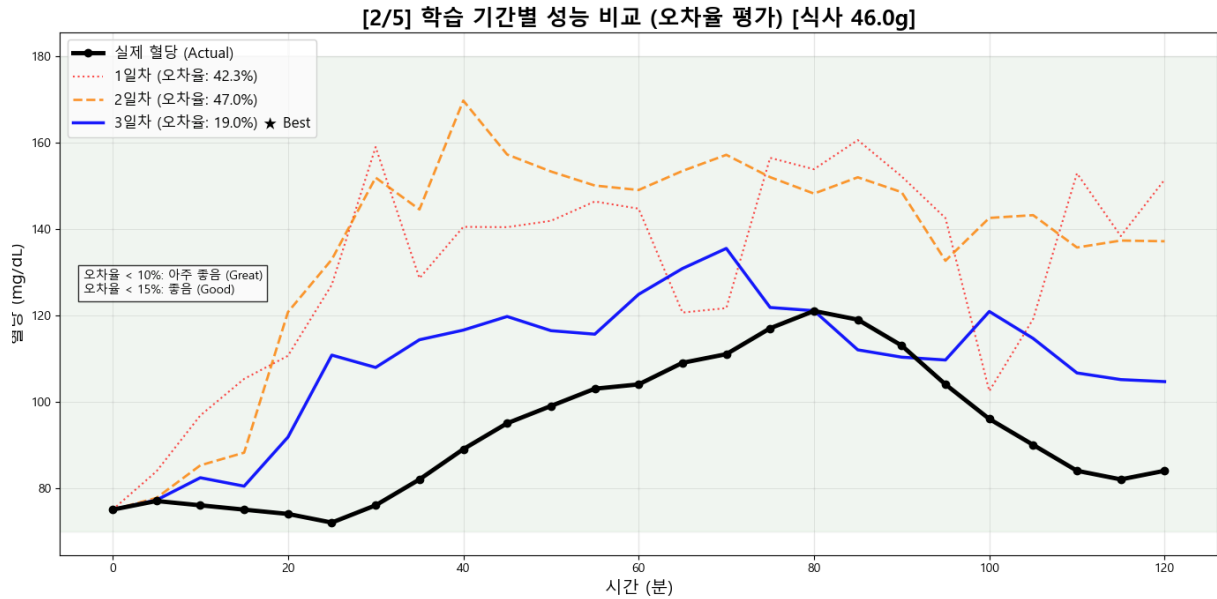
# 5. 저장 (Dump)
joblib.dump(model, model_path)

return {"status": "success", "message": f"User {user_id} 모델 저장 완료"}

```

3-5) 왜 XGBoost모델에 3일 이상의 데이터가 필요한가?

실험을 통해 유의미한 예측 값을 뽑기 위해서는 최소 4일 이상의 데이터가 필요했다.



설명 포인트 : 1,2일 학습의 경우 오차가 당뇨병학회가 말하는 유의미한 혈당 예측모델의 허용 오차율 15%를 넘어버리게 됨 >> 그래서 수학적 모델을 사용함 —> 기기기준이라 폐기

3-6) 응답 방법

모든 사용자(사용일자 상관x)는 식후 2시간의 혈당 변화 그래프를 받기 위해 2시간 동안 5분간의 포인트를 받게 됨

예측 모델 연산 요청시 필요한 데이터 형식

```
{
  "user_id": 101,
  "glucose": 138.5,
  "carbs_in": 60.0,
  "timestamp": "2026-01-30 17:00:00",
  "has_enough_data": true,
  "weight": 75.0,
  "height": 178.0,
  "is_t2d": true
}
```

24개의 포인트(120/5)를 리스트 형태로 반환 받게 됨

- 내부 구현 코드

```
class PredictionResponse(BaseModel):  
    status: str  
    method: Optional[str] = None  
    forecast: List[float] = []  
    message: Optional[str] = None
```

- 실제 출력 예시

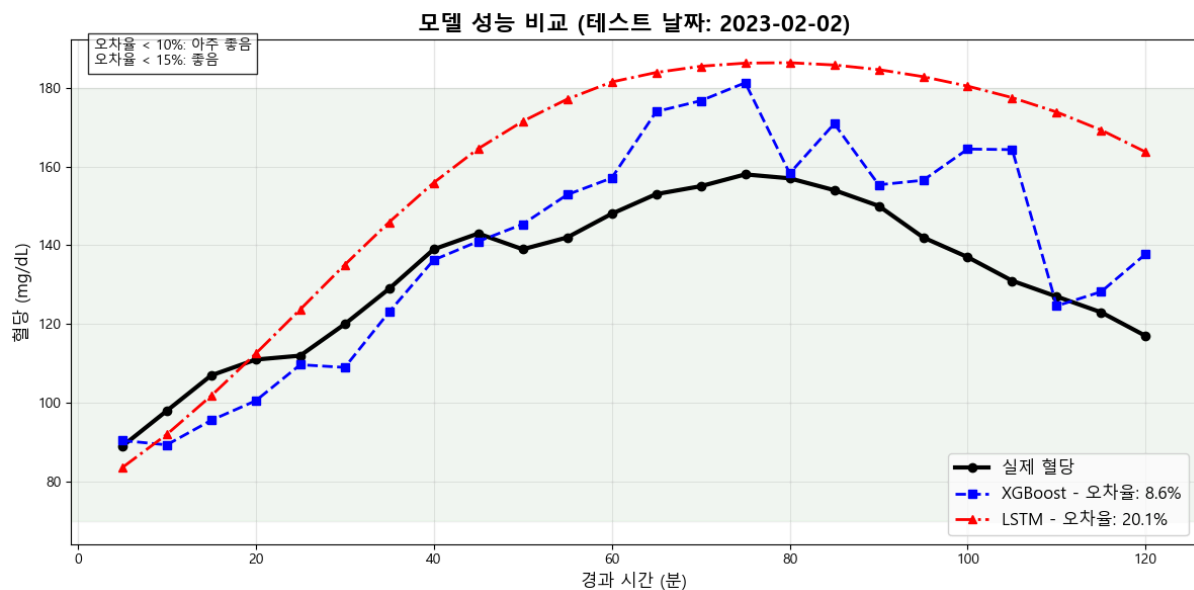
```
{  
    "status": "success",  
    "method": "math_engine",  
    "forecast": [  
        141.4,  
        147.0,  
        154.6,  
        163.4,  
        173.2,  
        183.4,  
        193.8,  
        204.1,  
        214.2,  
        224.0,  
        233.2,  
        241.9,  
        250.0,  
        257.5,  
        264.3,  
        270.5,  
        276.0,  
        281.0,  
        285.3,  
        289.1,  
        292.3,  
        295.1,  
        297.3,
```

```

299.1
],
"message": null
}

```

3-7) LSTM VS XGBoost 모델 비교



설명포인트

1. 오차 누적 방지

: LSTM 모델은 대표적인 재귀적 예측 모델 방식이기 때문에 초반에 한 번 틀리면 뒤로 갈수록 오차가 기하급수적으로 커지게 됩니다. 하지만 XGBOOST의 경우 MultiOutputRegressor를 사용해 미래 24개 포인트(2시간)를 독립적으로 한 번에 예측합니다. → 앞에서 틀려도 뒤쪽 예측에 영향을 주지 않아 오차율이 안정적입니다.

2. 혈당 데이터의 특성

: 혈당 데이터는 이미지나 자연어가 아닌 대표적인 정형 데이터입니다. 정형 데이터에서는 트리 기반 모델(XGBoost)이 딥러닝보다 성능이 우수하거나 대등하면서도 학습 효율이 훨씬 높습니다.

3. 데이터 수집의 한계

LSTM은 데이터의 패턴을 스스로 찾으려 하지만, 데이터 양이 적으면 엉뚱한 패턴을 배웁니다. 반면 XGBoost는 우리가 계산해준 COB(잔존 탄수화물), 가속도 같은 생리학적 변수를 직접적으로 활용합니다. 또한 데이터 양이 제한적인 상황에서도 LSTM보다 뛰어난 일반화 성능을 보였습니다.

4. 결론

:결과적으로 MARD(오차율) 기준으로 XGBoost는 약 8~9%대를 기록하며, LSTM(10~12%) 대비 약 2~3% 포인트 더 우수한 성능을 보였습니다.

4. 개선할 점

4-1

더 다양한 음식 사진을 인식할 수 있도록 추가학습 필요

4-2

더 많은 인자들을 반영할 수 있도록 관련 법안 및 데이터셋 확보 필요함. 특히 지금 서비스 타겟층이 2형 당뇨병 환자들을 위한 모델이기 때문에 인슐린 투약 정보를 반영하지 않았음.